



SMART PROPOSAL GENERATOR

ABSTRACT

The Smart Proposal Generator is an AI-driven system that uses Natural Language Processing (NLP) to automatically convert client briefs into structured, professional business proposals. Built using Python and Groq's Llama-3.1-8b-instant model, it analyzes unstructured text to identify key details such as objectives, timelines, budgets, and tone, then generates both JSON and formatted proposal drafts. The system also includes an evaluation module that measures tone consistency, structure completeness, and sentiment quality. Designed for automation and efficiency, this tool streamlines business communication and reduces the manual effort of proposal writing.

Usman Ali

Data Science Intern

Table of Contents

Smart Proposal Generator	2
1. Project Overview.....	2
2. Objectives.....	2
3. Key Features.....	2
4. Code Process Explanation (Step-by-Step)	3
5. NLP in the Project.....	4
6. Output Files.....	5
7. Flow Diagram	5
8. Conclusion.....	5

Smart Proposal Generator

1. Project Overview

The **Smart Proposal Generator** is an AI-based NLP system designed to automatically generate professional business proposals from raw client briefs.

It reads unstructured text (like a client’s request or project description), analyzes it using **Natural Language Processing**, and converts it into a **structured proposal** with clearly defined sections such as:

- Service Type
- Project Overview
- Objectives
- Timeline
- Estimated Budget
- Professional Proposal Text

The model integrates **Groq’s llama-3.1-8b-instant API** for intelligent text generation and **TextBlob** for NLP-based evaluation.

2. Objectives

1. Use NLP to understand and structure client requirements.
2. Automatically generate complete professional proposals.
3. Maintain consistency in tone, clarity, and structure.
4. Evaluate proposal accuracy, tone, and quality.
5. Build a modular, reusable Python system with clean documentation.
6. Allow users to control how many test cases they want to validate (minimum 10).

3. Key Features

Feature	Description
AI Proposal Generation	Converts client briefs into structured professional proposals.
Dynamic Test Control	User can choose how many test cases to run (min 10).
Random or Sequential Selection	User can pick test cases randomly or sequentially.
Automatic Evaluation	Rates each proposal for structure, tone, and sentiment.
Output Storage	Saves structured JSON and readable proposal text.
Error Handling	Detects invalid input and ensures minimum test case count.

Dataset Flexibility	Works with any dataset following required fields.
----------------------------	---

4. Code Process Explanation (Step-by-Step)

4.1. data_preprocessing.py

Purpose: Load and clean the client dataset.

Process:

1. Reads CSV file using pandas.
2. Removes empty rows.
3. Replaces missing values with "Unknown."
4. Returns a clean DataFrame for processing.

4.2. proposal_generator.py

Purpose: Generate proposals using NLP via Groq API.

Process:

1. Loads the **Groq API key** from .env.
2. Sends a structured prompt to the **LLaMA-3.1 model**.
3. The model performs NLP to:
 - o Understand the client brief.
 - o Extract key details (objectives, budget, timeline).
 - o Generate a well-formatted proposal.
4. The output is returned as a **JSON object**

4.3. utils.py

Purpose: Save outputs (text + JSON).

Functions:

- save_json(data, filename) → Saves structured proposals as JSON.
- append_text(text, filename) → Appends formatted proposals to a .txt file for easy reading.

4.4. evaluator.py

Purpose: Evaluate quality and tone of proposals using NLP.

Functions:

- `evaluate_proposal(proposal)` → Analyzes tone and structure using **TextBlob** sentiment analysis.
- `evaluate_all(proposals)` → Applies evaluation on all proposals and saves results.

Process:

1. Extracts text from each proposal.
2. Measures:
 - **Sentiment Polarity** → Tone quality.
 - **Structure Completeness** → Checks for missing fields.
 - **Overall Score** → Weighted average of tone, structure, and sentiment.
3. Saves evaluation results in `output/evaluation_report.json`.

4.5. `main.py`

Purpose: Run the full pipeline interactively.

Workflow:

1. Load and clean dataset.
2. Ask user how many test cases to run (minimum 10).
3. Ask user whether to choose **sequential** or **random** selection.
4. For each selected case:
 - Send to AI model for proposal generation.
 - Save text and JSON results.
5. Run evaluation for all proposals.
6. Print final success message and save outputs.

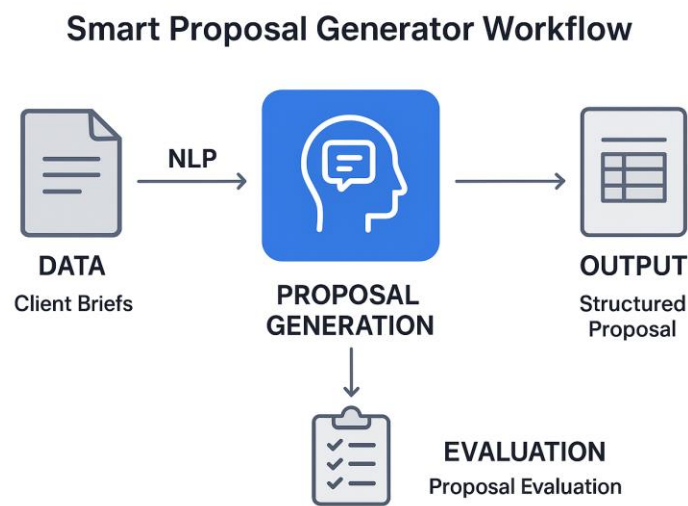
5. NLP in the Project

Stage	Module	NLP Function
Text Understanding	<code>proposal_generator.py</code>	Groq's LLM interprets raw client briefs.
Information Extraction	<code>proposal_generator.py</code>	Identifies service type, budget, objectives, etc.
Text Generation	<code>proposal_generator.py</code>	Produces coherent and professional proposal text.
Tone & Sentiment Analysis	<code>evaluator.py</code>	Uses TextBlob to analyze proposal tone.
Text Preprocessing	<code>data_preprocessing.py</code>	Cleans and normalizes textual input data.

6. Output Files

File	Description
proposal_samples.json	Structured AI-generated proposals.
generated_proposals.txt	Readable, formatted proposals.
evaluation_report.json	Evaluation results of all test cases.

7. Flow Diagram



8. Conclusion

The **Smart Proposal Generator** successfully meets all objectives:

- Uses **NLP** to transform natural client language into structured business proposals.
- Generates both **structured JSON** and **professional text output**.
- Automatically evaluates tone and structure.
- Provides a flexible and modular system ready for integration into a web frontend.

This system demonstrates a practical application of **AI and NLP** in automating real-world business communication workflows.